

Hands-On 1 - Window Functions

```
-- File: Window Functions
-- Developed by: WeCloudData

/*****
Create Database/Tables
*****/
-- Create the windowdb database if it doesn't exist
CREATE DATABASE IF NOT EXISTS windowdb;

USE windowdb;

-- Create a sales table if it doesn't exist
DROP TABLE IF EXISTS sales;

CREATE TABLE sales (
    name VARCHAR(50),
    month INT,
    sales INT
);

TRUNCATE sales;

-- Insert sample sales data for testing
INSERT INTO sales
VALUES
('james', 1, 200),
('james', 2, 300),
('james', 3, 400),
('james', 4, 150),
('james', 5, 100),
('james', 6, 200),
('james', 7, 350),
('james', 8, 300),
('james', 9, 400),
('james', 10, 200),
('james', 11, 250),
('james', 12, 350),
('kelly', 1, 400),
('kelly', 2, 300),
('kelly', 3, 500),
('kelly', 4, 250),
('kelly', 5, 450),
('kelly', 6, 300),
('kelly', 7, 300),
('kelly', 8, 350),
('kelly', 9, 400),
('kelly', 10, 300),
('kelly', 11, 250),
('kelly', 12, 350);

SELECT * FROM sales;

/*****
Window Function Introduction
*****/

/*
The keyword `OVER` signals that this is a `window function`, as opposed to a `grouped aggregate function`.

- The empty parentheses after `OVER` is a window specification.
- In this simple example, it is empty `()` meaning it aggregates the window function over all rows in the result set.
*/

SELECT SUM(sales) AS total_sales
FROM sales;
```

```

SELECT name, month, sales, SUM(sales) OVER () AS total_sales
FROM sales;

```

```

/*****
Window - Partitioning
*****/

```

```

/*
Example 1: Calculate total sales for each salesperson
*/

```

```

-- Get total sales for each salesperson
SELECT name, SUM(sales)
FROM sales
GROUP BY name;

```

```

-- Partition the data by salesperson and calculate total sales
-- WRITE YOUR QUERY HERE

```

```

/*
Example 2: Calculate total sales for each month
*/

```

```

-- Get total sales for each month
SELECT month, SUM(sales)
FROM sales
GROUP BY month;

```

```

-- Partition the data by month and calculate total sales
-- WRITE YOUR QUERY HERE

```

```

/*****
Window - ORDER BY
*****/

```

```

/*
Calculate the cumulative sum of sales for each salesperson
*/
-- WRITE YOUR QUERY HERE

```

```

/*****
EXERCISE
*****/

```

```

USE superstore;

```

```

/*
1. Calculate the cumulative sum of sales by month for the year 2011
Hint: Apply knowledge of subquery and subquery
*/

```

```

-- WRITE YOUR QUERY HERE

```

```

/*
2. Calculate the cumulative sum of sales by month for each year (2009, 2010, 2011, 2012)
Hint: Apply knowledge of subquery and subquery
*/

```

```

-- WRITE YOUR QUERY HERE

```

```

/*****
Movable Windows
*****/

```

```

USE windowdb;

```

```

/*
Calculate moving total sales (1 preceding, 1 following)
*/

```

```

SELECT
*,
SUM(sales) OVER (
ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING
) AS moving_window
FROM sales
WHERE name = 'james';

```

```

/*
Calculate moving average sales (1 preceding, 1 following)
*/
-- WRITE YOUR QUERY HERE

```

```

/*****
EXERCISE
*****/

```

```

USE superstore;

/*
  Calculate the moving average (2 preceding, 2 following) of total sales per month for each year
*/
-- WRITE YOUR QUERY HERE

/*****
  Special Window Functions
  *****/
USE windowdb;

/*
  ROW_NUMBER() - Assigns a unique row number to each row within a partition
*/
SELECT
  name,
  month,
  sales,
  ROW_NUMBER() OVER (PARTITION BY name ORDER BY sales DESC) AS row_num
FROM sales;

/*
  RANK() - Assigns a rank to each row within a partition, with gaps for ties
*/
SELECT
  name,
  month,
  sales,
  RANK() OVER (PARTITION BY name ORDER BY sales DESC) AS rank_sales
FROM sales;

/*
  DENSE_RANK() - Assigns a rank to each row within a partition without gaps for ties
*/
SELECT
  name,
  month,
  sales,
  DENSE_RANK() OVER (PARTITION BY name ORDER BY sales DESC) AS dense_rank_sales
FROM sales;

/*
  LEAD() - Accesses data from the next row in the result set
*/
SELECT
  name,
  month,
  sales,
  LEAD(sales) OVER (PARTITION BY name ORDER BY sales DESC) AS next_month_sales
FROM sales;

/*
  LAG() - Accesses data from the previous row in the result set
*/
SELECT
  name,
  month,
  sales,
  LAG(sales) OVER (PARTITION BY name ORDER BY sales DESC) AS prev_month_sales
FROM sales;

/*
  NTILE() - Divides the result set into a specified number of buckets (quartiles in this case)
*/
SELECT
  name,
  month,
  sales,
  NTILE(4) OVER (PARTITION BY name ORDER BY sales DESC) AS sales_quartile
FROM sales;

/*****
  EXERCISE
  *****/
USE windowdb;

/*
  1. Calculate average sales by quarter for each salesperson
  This involves using NTILE to divide months into quarters, grouping by salesperson, and calculating average
  sales
  Hint: What you might need, in no particular order: NTILE() Window Function, Group By, Subquery
*/

```

```
-- Expected Output:
-- Name, quarter, avg(sales)
-- james, 1, 300
-- james, 2, 150
-- ...
-- kelly, 4, 300
```

```
SELECT * FROM sales;
```

```
-- WRITE YOUR QUERY HERE
```

```
/******
```

EXERCISE

```
******/
```

```
-- Problem: We need to calculate the cumulative sales for each month in the year 2011
```

```
-- 1. Filter the data to only include orders from 2011.
```

```
-- 2. Calculate the total sales for each month.
```

```
-- 3. Compute the cumulative sum** of monthly sales, which represents the running total of sales from January to December.
```

```
-- This analysis helps observe sales trends over the course of the year, identifying seasonality and performance patterns.
```

```
USE superstore;
```

```
-- WRITE YOUR QUERY HERE
```

```
/******
```

EXERCISE - Interview Question

```
******/
```

```
-- Drop the existing retail database if it exists
```

```
DROP DATABASE IF EXISTS retail;
```

```
CREATE DATABASE retail;
```

```
USE retail;
```

```
-- Create the retail_promo table
```

```
CREATE TABLE retail.retail_promo (
```

```
    user_id INT,
```

```
    transaction_id INT,
```

```
    order_date DATE,
```

```
    order_sales FLOAT,
```

```
    coupon_activation CHAR(1)
```

```
);
```

```
TRUNCATE retail.retail_promo;
```

```
-- Insert sample data into retail_promo table
```

```
INSERT INTO retail.retail_promo (user_id, transaction_id, order_date, order_sales, coupon_activation)
VALUES
```

```
    (1, 485948, '2018-01-02', 20.89, 'N'),
```

```
    (1, 217493, '2018-01-03', 10.11, 'Y'),
```

```
    -- (1, 23, '2018-01-04', 10.11, 'Y'), -- In the case if there was a user who activated the coupon twice in a row
```

```
    (1, 732164, '2018-01-05', 10.00, 'N'),
```

```
    (1, 327146, '2018-01-10', 15.34, 'N'),
```

```
    (2, 483938, '2018-01-01', 17.89, 'Y'),
```

```
    (2, 347683, '2018-01-06', 10.00, 'N'),
```

```
    (3, 458792, '2018-01-06', 5.00, 'N'),
```

```
    (3, 112893, '2018-01-07', 15.34, 'Y');
```

```
SELECT * FROM retail_promo;
```

```
/*
```

Company X launched a marketing campaign. Customers were given coupons that they can use in the future trips. The marketing team wants to know how many users actually made purchases after coupon activation.

```
*/
```

```
/*
```

Use the LEAD() window function to check if a purchase occurred after coupon activation

```
*/
```

```
-- WRITE YOUR QUERY HERE
```